Session 2

NestJS Framework

Objectives

By the end of this session, You will be able to:

1. Understand why NestJS exists
2. Learn the difference: Node.js vs Express vs NestJS
3. Master Decorators
4. Understand Separation of Concerns
5. Implement Middleware & Guards

Table of contents

01

Recap

02

The Evolution & Why NestJS

03

Node.js vs Express vs NestJS

04

NestJS Philosophy

05

Understanding Decorators

06

Separation of Concerns

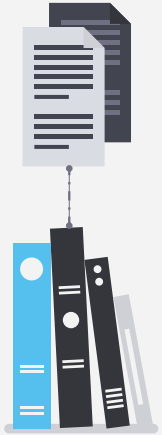


Table of contents

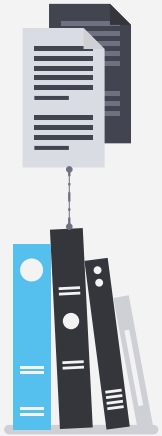
07 Module System Architecture

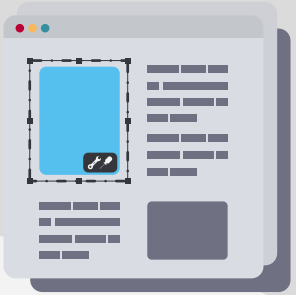
08 Middleware

09 Guards

10 Pips

11 Interceptors





01 Recap

- What happened last Session?
- What did we talk about?
- What was the benefit you got from last session?

Table of Content

1_ Problem with JS

2_ How the Problem is Solved

3_ What is TypeScript

4_ Basics of TypeScript

- ★ Types
- ★ Functions
- ★ Interfaces
- ★ Union Types
- ★ Intersection Types
- ★ Generic Types
- ★ Any
- ★ Type vs Interface vs Class vs Object

Table of Content

5_ code Structure Different Philosophies

6_ How TS is Transpiled

7_ How to set it up

8_ What Files Matters to us

Table of contents

~~01~~

~~Recap~~

02

The Evolution & Why NestJS

03

Node.js vs Express vs NestJS

04

NestJS Philosophy

05

Understanding Decorators

06

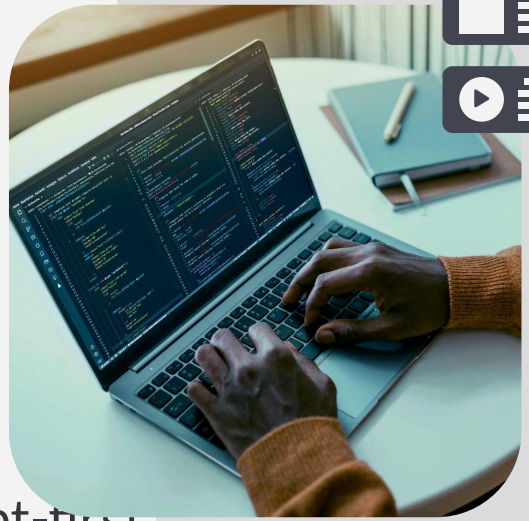
Separation of Concerns

02

The Evolution & Why NestJS

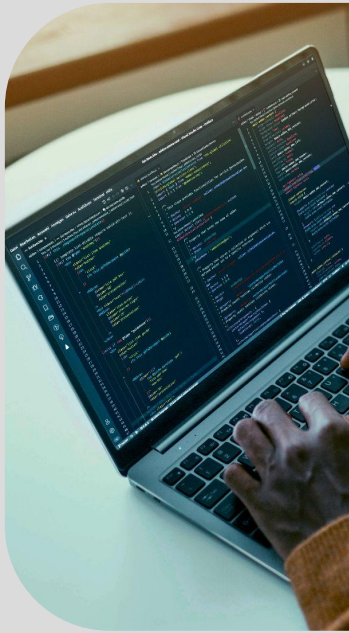
The Node.js Framework Timeline

- 2010: Express.js → Minimalist, unopinionated, flexible
- 2013: Koa.js → Async/await focus
- 2011: Hapi.js → Configuration-centric
- 2016: Fastify → Performance-first
- 2017: NestJS → Enterprise, opinionated, TypeScript-first



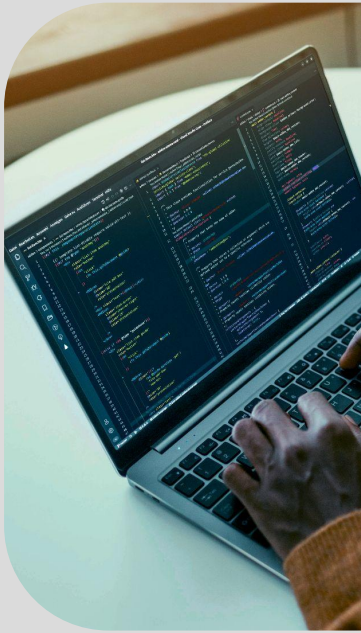
Express Problems at Scale

- As **projects** grow, Express applications often face several challenges:
- No enforced structure → Code becomes messy
- No standard patterns → Every developer does it differently
- Manual dependency management → Hard to test
- Mixing concerns → Business logic in route handlers



What is NestJS

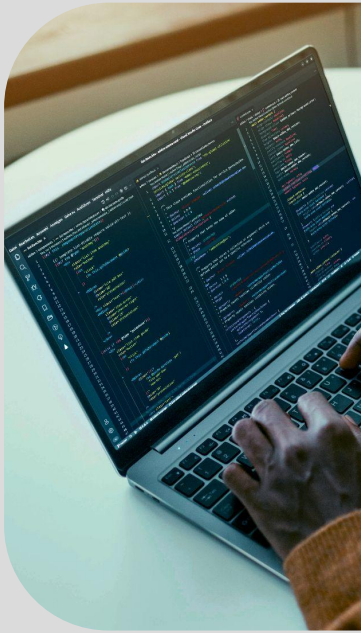
- a progressive opinionated Node.js framework for building efficient, scalable, and reliable server-side applications using TypeScript or JavaScript



Why NestJS Was Created

NestJS solves these problems by providing:

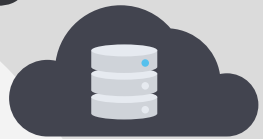
- Structure Chaos → Enforced modular architecture
- Scalability Issues → Built for large teams and projects
- Testing Overhead → Built-in testing utilities
- Dependency Hell → Automatic dependency injection
- Inconsistent Patterns → Standardized approach



03

Node.js vs Express vs NestJS

Node.js vs Express vs NestJS



Understanding the Layers

- NestJS Application Code (Top Layer)
- NestJS Framework
- Express (or Fastify) Framework
- Node.js Runtime
- V8 JavaScript Engine (Bottom Layer)

Node.js vs Express vs NestJS

Node.js

- Runtime environment (like Java JVM)

Express

- Minimalist web framework

NestJs

- Opinionated framework built on Express/Fastify

Node.js vs Express vs NestJS

Aspect	Express	NestJS
Structure	None	Enforced modules
Dependencies	Manual imports	Dependency Injection
TypeScript	Optional	First-class
Testing	Manual setup	Built-in
Learning Curve	Easy	Steeper
Best For	Small apps	Enterprise apps

04

NestJS Philosophy

Core Principles

- ## TypeScript-First

  NestJS is built with and for TypeScript, catching errors at compile-time rather than runtime.

- ## Dependency Injection

  The framework manages dependencies for you, making code cleaner and easier to test.

- ## Modularity

  Code is organized into feature modules (e.g., Users, Products), keeping related code together.

- ## Platform Agnostic

  NestJS can run on top of Express (default) or Fastify, allowing you to switch underlying frameworks easily.

Architectural Inspirations

- ★ ***Angular*** → Module system, DI, decorators
- ★ ***Spring (Java)*** → Enterprise patterns, annotations
- ★ ***ASP.NET Core*** → Middleware pipeline, service registration

Table of contents

01 Recap

02 The Evolution & Why NestJS

03 Node js vs Express vs NestJS

04 NestJS Philosophy

05 Understanding Decorators

06 Separation of Concerns

05

Understanding Decorators

What Are Decorators?

- ***Definition:***
  Functions that attach metadata to classes, methods, or parameters. They are the "magic" that makes NestJS work.

- ***Think of them as:***
  Labels or tags that give special meaning to code.

Types of Decorators

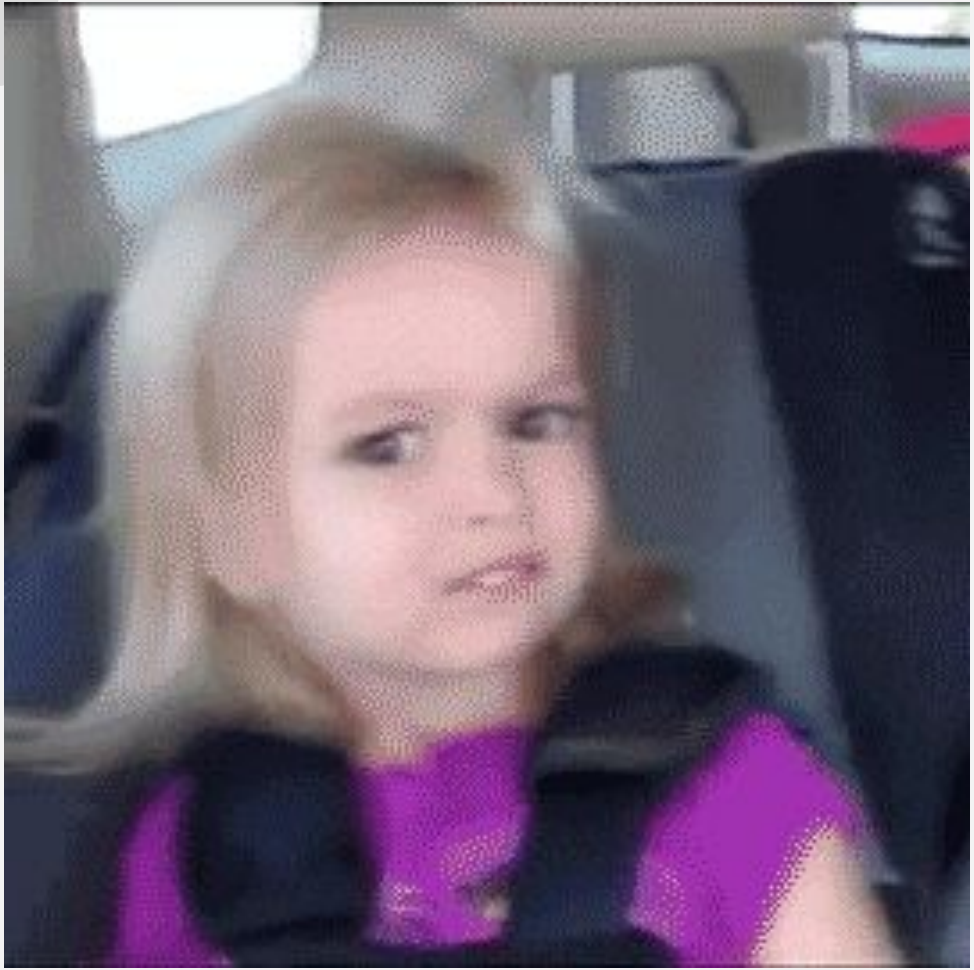
1. ***Class Decorators:*** Mark a class as a Controller, Service, or Module.

2. ***Method Decorators:*** Define HTTP methods (Get, Post) for functions.

3. ***Parameter Decorators:*** Extract data from the request (Body, Param, Query).

Why Decorators Matter

- Route definitions live with handlers
- Type-safe parameters
- Self-documenting
- Framework can analyze structure



06

Separation of Concerns

The Three Layers

Separation of Concerns is a software design principle that says:

"Each part of your application should have a single, well-defined responsibility."

- Instead of mixing everything together, you split your code into layers, so each layer only does one thing.
- This makes your code: Clean, Maintainable, Testable, and Scalable .

The Three Layers

1. ***Controllers:***

   HTTP concerns only. Handle requests and send responses.

2. ***Services/Providers:***

   Business logic. Where the actual work happens.

3. ***Data Access:***

   Database operations.

The Three Layers

Controllers: HTTP Layer

Responsibility: Handle HTTP requests and responses ONLY.

Should NOT: Contain business logic, make database queries, or perform calculations.

Services: Business Logic Layer

Responsibility: Implement business rules and logic.

The @Injectable() Decorator: Marks a class as managed by the NestJS container.

Table of contents

01 Recap

02 The Evolution & Why NestJS

03 Node js vs Express vs NestJS

04 NestJS Philosophy

05 Understanding Decorators

06 Separation of Concerns

Table of contents

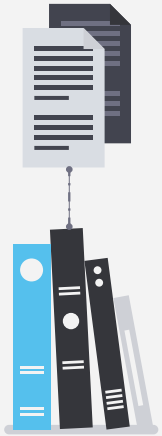
07 Module System Architecture

08 Middleware

09 Guards

10 Pips

11 Interceptors



07

Module System Architecture

What Are Modules?

Definition:

Organize related components (controllers, services) into cohesive blocks. Every NestJS application has at least one module (the root module).

Module Relationships

- Feature Modules: Encapsulate specific features (e.g., UserModule).
- Shared Modules: Share services (like DatabaseService) across the application.
- Global Modules: Available everywhere without importing (e.g., LoggerModule).

08

Middleware

What is Middleware?

Definition:

Functions that execute before the route handler. They sit between the request and the controller.

Common Uses:

- Logging
- Parsing request bodies
- Authentication (basic)

Types in NestJS:

- Functional Middleware: Simple functions.
- Class-based Middleware: Classes that can inject dependencies.

09

Guards

What Are Guards?

Definition:

Determine if a request should proceed based on conditions (auth, roles, etc.).

Key Difference from Middleware:

- Guards run after middleware
- Have access to execution context (knows what handler will be called)
- Specifically for authorization decisions

What Are Guards?

Common Guards:

- AuthGuard: Checks if a user is logged in.
- RolesGuard: Checks if a user has the required permission.

10

pips

What are Pips?

Its job is to process incoming data before it reaches the controller.

Key points:

- validate, transform, or sanitize request data.
- Keeps controllers clean, because all input processing is handled here.

11

Interceptors

What are Interceptors?

An Interceptor is a class that wraps around a controller method. Can run before and after the controller executes.

Key points:

Can log requests, measure execution time, cache responses, or transform output. Can inject other services/providers using DI.

Execution Order:

Request → Middleware → Guards → Interceptors (Before) → Pipes → Controller → Interceptors (After) → Service → response

When to Use What

Use Case	Tool
Logging requests	Middleware
Parsing body	Middleware
Authentication	Guard
Authorization (roles)	Guard
Validation	Pipe
Transform response	Interceptor

Table of contents

07 Module System Architecture

08 Middleware

09 Guards

10 Pips

11 Interceptors

Big Announcement!

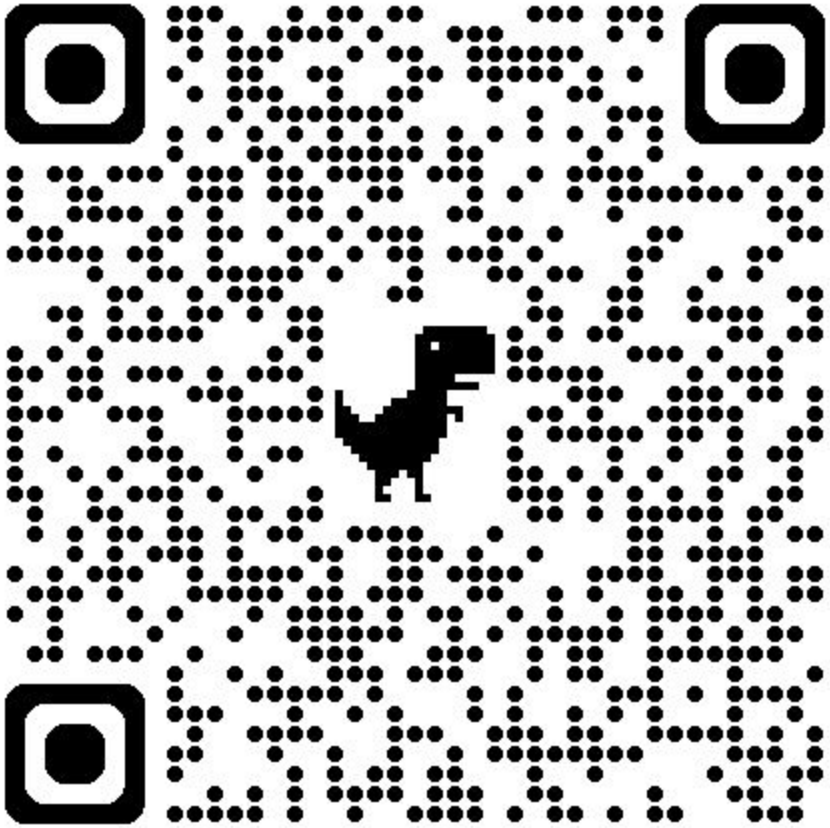
Who is best of the Week?

Game Time!

Mohsens Edition!

من سيربح المليون

Game!



1_ استعين بصديق

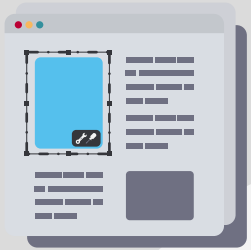
2_ حذف اجابتين

3_ تصويت الجمهور

theory mode: completed!

Time to roll up our sleeves and dive into the fun part

the technical part begins



Thanks!